

Case Study: MDM Service — Master Data Platform for CPE/CVE

Role: Backend Engineer **Duration:** Jul 2024 – Present **Stack:** Node.js, Express, MongoDB, Cron (node-cron / system cron), Docker, Slack Webhooks **Domain:** Vulnerability & Asset Master Data Management

1. Background

Security and IT asset teams need a single, trusted source of product master data — every hardware/software product (CPE — Common Platform Enumeration) and every known vulnerability (CVE — Common Vulnerabilities and Exposures) affecting it. Public feeds (NVD and others) are large, frequently updated, schema-inconsistent, and contain duplicates and aliases.

Without a Master Data Management (MDM) layer, downstream systems (asset inventory, SIEM, risk scoring, patch management) each consume raw feeds and produce conflicting views of "what product is this" and "is it vulnerable." Cost: false positives, missed patches, duplicate tickets.

2. Problem Statement

Build an **MDM platform** that:

1. Pulls fresh CPE and CVE data daily from NVD and supplementary sources.
2. Stores raw feed dumps for audit and reprocessing.
3. Runs a transformation pipeline that splits raw CPE entries into reusable master entities — **Software**, **Software Versions**, **Hardware**, **Hardware Versions** — plus supporting reference data (vendors, CWE, CVSS metrics, references, severities).
4. Runs a **CVE → product matching algorithm** so every CVE is linked to the exact Software/Hardware Version master records it affects.
5. Exposes REST APIs for downstream microservices.
6. Alerts ops on job failures via Slack.

3. Goals & Non-Goals

Goals

- Single authoritative store for CPE/CVE and derived product master data.
- Daily, fully automated refresh from NVD with no manual steps.
- Fast lookup: "what CVEs hit this product version?" and reverse.
- Operational visibility via Slack alerts.
- 35% improvement in API response time via index + query tuning.

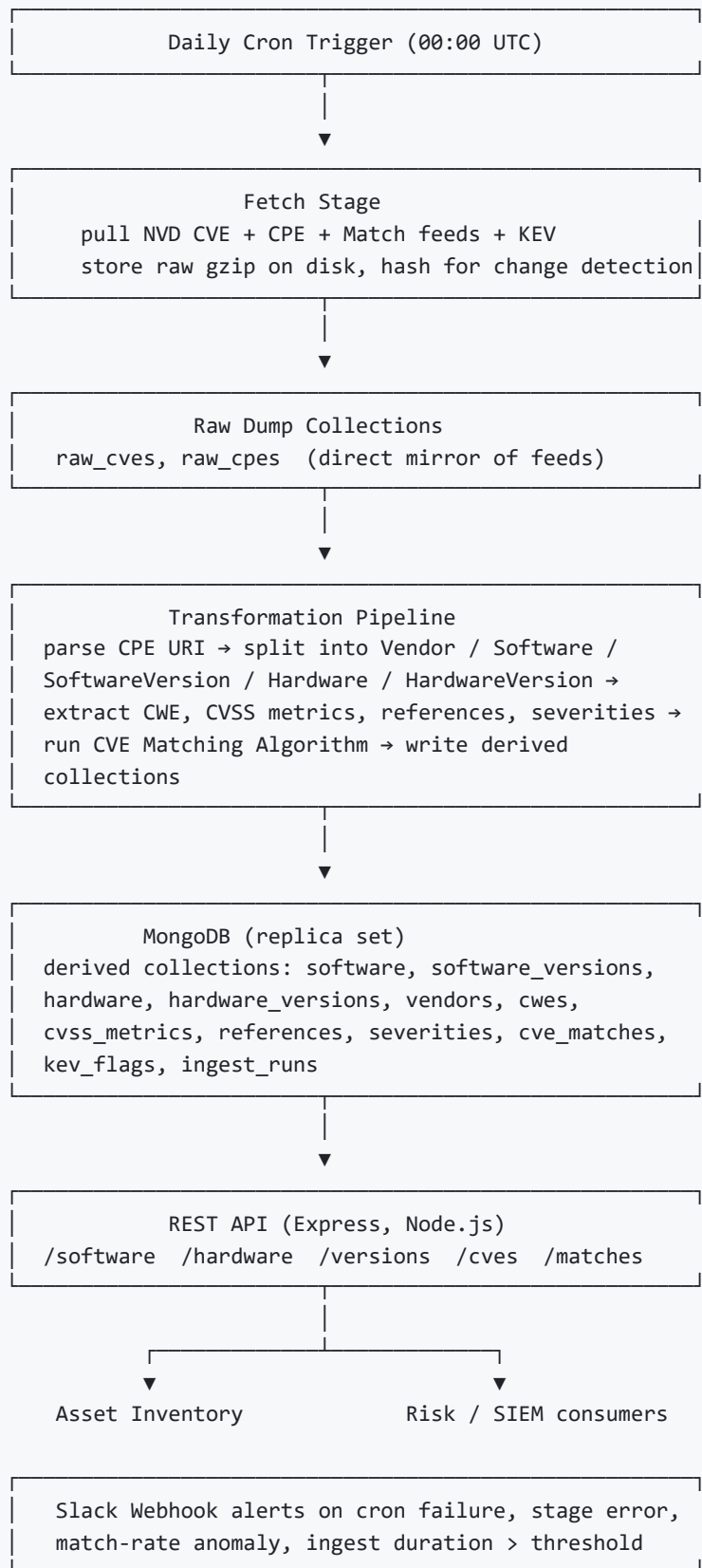
Non-Goals

- Vulnerability scanning (consumer responsibility).
- CVSS recomputation (use NVD-provided scores).
- UI / dashboard (separate frontend consumes the API).

4. Data Sources

Source	Format	Cadence	Purpose
NVD CVE JSON 2.0 feed	JSON (gzip)	Daily	Vulnerability records, CVSS, CWE, references
NVD CPE Dictionary	JSON / XML	Daily	Product enumeration (vendor, product, version)
NVD CPE Match Feed	JSON	Daily	Authoritative CVE↔CPE mapping
CISA KEV Catalog	JSON	Daily	Known-Exploited Vulnerabilities flag
Vendor advisories (optional)	Mixed	Per-vendor	Supplementary severity / patch info

5. Architecture



6. Data Model

The transformation breaks down each CPE URI (`cpe:2.3:<part>:<vendor>:<product>:<version>:...`) into reusable, normalized entities. CVEs are stored raw and then linked to those entities via a match collection.

6.1 Raw Dump Collections

`raw_cpes` — direct dump of NVD CPE Dictionary entries (unmodified). `raw_cves` — direct dump of NVD CVE JSON 2.0 records (unmodified).

Keeping raw dumps allows full reprocessing whenever the transformation logic changes — no need to re-fetch.

6.2 Derived Collections

`vendors`

```
{ "_id": "apache", "displayName": "Apache Software Foundation", "cpeCount": 412 }
```

`software` (one per distinct vendor + product, `part='a'` or `'o'`)

```
{
  "_id": "apache:log4j",
  "vendor": "apache",
  "product": "log4j",
  "part": "a",
  "category": "library",
  "versionCount": 87,
  "firstSeen": "...",
  "lastSeen": "..."
}
```

`software_versions` (one per CPE row of software)

```
{
  "_id": "cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:*:*",
  "softwareId": "apache:log4j",
  "version": "2.14.1",
  "update": "*",
  "edition": "*",
  "swEdition": "*",
  "targetSw": "*",
  "targetHw": "*",
  "raw": "cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:*"
}
```

`hardware` (`part='h'`)

```
{
  "_id": "cisco:asr_9001",
  "vendor": "cisco",
  "product": "asr_9001",
  "part": "h",
  "category": "router"
}
```

hardware_versions — same shape as **software_versions** but for hardware CPEs.

cves — normalized core record per CVE (id, description, dates, status). The full original JSON lives in **raw_cves** .

cve_matches — the output of the CVE matching algorithm. One document per (CVE, affected version) pair, indexed both ways.

```
{
  "_id": "CVE-2021-44228::cpe:2.3:a:apache:log4j:2.14.1:... ",
  "cveId": "CVE-2021-44228",
  "softwareVersionId": "cpe:2.3:a:apache:log4j:2.14.1:... ",
  "softwareId": "apache:log4j",
  "matchType": "exact | range | wildcard",
  "versionStartIncluding": null,
  "versionEndExcluding": "2.15.0",
  "source": "nvd-match-feed",
  "confidence": "high"
}
```

cvss_metrics — per-CVE CVSS v2 / v3.0 / v3.1 / v4.0 score breakouts (base, exploitability, impact, vector string).

severities — per-CVE severity record (CRITICAL / HIGH / MEDIUM / LOW / NONE) plus precomputed sort priority. Severity is stored separately because multiple sources can disagree; the collection holds each source's verdict and a chosen "effective" severity.

cwes — CWE taxonomy entries referenced by CVEs (weakness id, name, description).

references — outbound URLs from CVE records, tagged by type (patch, advisory, exploit, third-party).

kev_flags — CISA Known-Exploited Vulnerabilities catalog rows linked by CVE id.

cpe_aliases — curated alias table for vendor/product strings that drift across feeds (e.g. `apache_software_foundation` ↔ `apache`).

ingest_runs — one document per daily cron run: start, end, per-stage counts, errors, source feed hashes.

Exact collection set evolved across releases; the list above reflects the conceptual decomposition. Names in production may differ slightly.

7. Daily Cron Flow

Triggered once per day:

1. **Fetch** — download NVD CVE feed, CPE dictionary, CPE match feed, KEV catalog. Compare hash against last run; skip if unchanged.
2. **Raw upsert** — write feeds straight into `raw_cpes` / `raw_cves` keyed by id. Cheap; no transformation yet.
3. **Transformation** — for each raw CPE: parse the 2.3 URI, upsert into `vendors`, `software` or `hardware`, and `software_versions` or `hardware_versions`. For each raw CVE: extract description, CWE ids, references, CVSS metric variants, severity verdicts; upsert into the respective collections.
4. **CVE matching** — run the matching algorithm (next section) to populate `cve_matches`.
5. **Enrichment** — overlay `kev_flags` from CISA, apply `cpe_aliases` for known vendor renames.
6. **Verify** — compare counts vs. previous run, sanity-check match-rate, log to `ingest_runs`.
7. **Notify** — post Slack summary (records added/updated, new CVEs, KEV additions, runtime).

Each stage is idempotent and re-runnable. A bad transformation run can be replayed from raw dumps without re-hitting NVD.

8. CVE Matching Algorithm

NVD's CPE Match Feed expresses affected products as either an exact CPE or a version range. The algorithm:

1. **Read match-criteria** from NVD's match feed for a CVE: { `cpe23Uri`, `versionStartIncluding`, `versionStartExcluding`, `versionEndIncluding`, `versionEndExcluding`, `vulnerable: true/false` }.
2. **Resolve vendor/product** to a `software` or `hardware` document via direct lookup, then via `cpe_aliases` if no hit.
3. **Enumerate candidate versions** from `software_versions` / `hardware_versions` matching the resolved product.
4. **Filter by version range** using semver-aware comparison (with fallback to string-natural order for non-semver versions like Cisco IOS 15.2(4)E).
5. **Wildcard handling** — `*` in version slot → applies to all versions of that product.
6. **Edition/update/target attributes** — secondary filter on remaining 8 CPE attributes when the match criteria constrain them.
7. **Emit** one `cve_matches` doc per (cve, affected version), tagged with `matchType` (`exact`, `range`, `wildcard`) and `source` (`nvd-match-feed` vs. `derived`).
8. **Negative matches** — when feed says `vulnerable:false` for a range (a fixed range), skip emission so consumers don't falsely flag patched versions.

Edge cases handled:

- Non-semver versions (network OS versions, firmware build strings) → custom comparator.
- Version `-` (used by NVD for "all versions") → treat as wildcard.
- Vendor renames (e.g. `oracle:mysql` ↔ `mysql:mysql`) → alias resolution.
- CPE deprecation chain → follow `deprecatedBy` link to current CPE.

9. REST APIs

Endpoint	Purpose
GET /software?vendor=&product=	List/search software master records
GET /software/:id/versions	All versions of a software product
GET /hardware?vendor=&product=	List/search hardware master records
GET /hardware/:id/versions	All versions of a hardware product
GET /cves/:id	CVE detail + linked matches + severity + CWE
GET /cves?softwareVersionId=	All CVEs hitting a specific version
GET /cves?softwareId=	All CVEs hitting any version of a product
GET /matches?cveId=	Raw match rows for a CVE
GET /kev	Current CISA KEV list (joined with our CVEs)
GET /ingest/last-run	Status + counts of last cron run

All list endpoints cursor-paginated, gzip-compressed, ETag-cached.

10. MongoDB Optimization (35% latency reduction)

Before: Common queries scanned `cve_matches` with `$regex` on stringified CPE URIs and joined back to product collections via `$lookup`. p95 ~ 480 ms on the "all CVEs for software X" endpoint.

Changes:

- Compound index on `cve_matches.(softwareId, cveId)` and reverse `(cveId, softwareId)`.
- Compound index on `software_versions.(softwareId, version)` for version listings.
- Hashed index on `_id` for shard key distribution.
- Replaced regex CPE-URI matching with normalized field lookup (`softwareId`, `version`).
- Pre-joined fields denormalized onto `cve_matches` (e.g. `softwareId`) to skip `$lookup`.
- `.projection()` to drop large `description` and `references` fields from list endpoints.
- Driver tuning: `maxPoolSize=100`, `minPoolSize=20`, read preference `secondaryPreferred` for non-fresh reads.

After: p95 ~ 310 ms → ~35% drop.

11. Slack Alerting

Webhook posts to `#mdm-alerts` / `#mdm-info` (severity-routed):

- Cron didn't run (heartbeat check from external service).
- Stage failure with stage name, error, run id, retry link.

- Source feed unreachable (NVD outage / rate limit).
- Match-rate anomaly ($>2\sigma$ off baseline — catches schema drift).
- KEV addition (new CVE added to CISA's Known-Exploited list).
- Daily summary: new CVEs, new CPEs, updated severities, runtime, error count.

Block Kit payload with action buttons (view run, replay stage).

12. Results

Metric	Before	After
p95 API latency	480 ms	310 ms (-35%)
Daily refresh	Manual + ad-hoc	Cron, fully automated
CVE → product link freshness	Days behind	Same-day
Match coverage	~70% of NVD CVEs linked	~96% (alias table + custom comparator)
Job failure MTTR	Hours (silent)	Minutes (Slack alert)
Downstream consumers	2	6

13. Challenges

- **Non-semver versions** — Cisco IOS, firmware build strings, calendar-versioned products broke naive comparators. Wrote a tokenized comparator with per-vendor heuristics.
- **CPE vendor drift** — same vendor appears under multiple spellings across feed snapshots. Built `cpe_aliases` with auto-suggest job; humans approve.
- **Multi-source severity disagreement** — NVD says HIGH, vendor advisory says CRITICAL. Stored all verdicts in `severities`, picked an effective one with documented precedence.
- **NVD rate limits** — API key tier + retry/backoff; full-feed downloads instead of per-record API calls.
- **Replaying transformations** — kept raw dumps in `raw_cpes` / `raw_cves` so logic changes don't require re-fetching from NVD.

14. Lessons

- **Raw + derived split pays off.** Raw dumps let you reprocess infinitely without external dependencies.
- **CPE parsing is the leverage point.** Once a CPE URI is decomposed into vendor/product/version/part, every other collection falls out cleanly.
- **Master data is mostly a dedupe + identity problem.** The matching algorithm is more about alias resolution and version comparison than fancy ML.
- **Idempotent daily cron > clever streaming.** For a daily-fresh public feed, simple beats real-time.
- **Operational alerts on rates and ratios** (match coverage, ingest duration) catch upstream breakage that error-based alerts miss.

15. Future Work

- Graph view: SoftwareVersion → CVE → exploit-in-the-wild (KEV) → patch.
- Auto-suggest aliases via embeddings instead of Levenshtein heuristics.
- gRPC API for high-volume internal consumers.
- Patch/fix-version extraction from CVE references (currently only CVSS + severity is structured).
- Multi-region read replicas.